

The moment a vehicle comes within four metres distance, the camera takes a frontal picture of the vehicle and compares its registration plate number with the already available numbers in its database.

HC-SR04 cannot measure beyond four metres. To make it measure



Fig. 3: LCD display



Fig. 4: Ultrasonic sensor



Fig. 5: Raspberry Pi

beyond that, say, up to six metres, it can be fitted on a two-metre long pole with wires in front of the camera on the side of the passage such that when the barrier opens, it will go up with the barrier.

You can use voice synthesizer 'espeak' to issue voice instructions to the drivers. The instructions can be like welcome, slow down, come closer, drive away, etc.

To increase range of measurement further, from six metres to twelve metres, a TFMini-S (3.4-degree aperture) camera can be used, though a four to six metres distance is generally just right for this kind of activity. In case of an IR tag system, the effective distance is about two metres only from the scanner. That's why the vehicle drivers are often asked to move closer.

If anyone thinks that by holding a registration plate in hand one can fool the system, that is not possible at all. The car classifier model "haarcascade_russian_plate_number.xml" is

used here to identify the car first and then the number plate is identified, scooped, and read by this system.

The author's prototype is shown in Fig. 1. The components needed for the project are listed under the Bill of Material table. Fig. 2 shows the circuit diagram of the project.

Here, for testing the project, an LED is being used as an indication for the stop barrier's working. In real deployment, you can use a relay and a linear actuator to open the gate barrier.

As shown in Fig. 2, connect the HDMI display to the Raspberry Pi HDMI and the ultrasonic sensor to Raspberry Pi GPIO pin. Then connect the camera to the camera port using the ribbon cable provided with the camera. You can connect the actuator LED to the pin for the gate open or close actions. The images of the LCD, the ultrasonic sensor, and the camera are shown in Fig. 3, Fig. 4, and Fig. 5, respectively.

The software for this project is prepared in Python IDLE and mod-

```
import cv2
import pytesseract as tess
from gpiozero import LED
from time import sleep
gate = LED(21)

#UPDATE YOUR DATABASE IN THE FOLLOWING STRING
carpool=['OR06F4010','UP16BY3569','MP13CA7644','MH03AW8444','MH03AW8444']
def check(image_file):
    #carpool=['OR06F4010','UP16BY3569','MP13CA7644','MH03AW8444']
    width1 = [650,660,670] #560,570,580,590,600,610,620,630,640,650,660,670] #670 800
    height1 = 475 #475
    for wd in width1:
        # load the image, resize it, and convert it to grayscale
        image = cv2.imread(image_file) #6.jpeg")
        image = cv2.resize(image, (wd, height1))
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # load the number plate detector
    n_plate_detector = cv2.CascadeClassifier("haarcascade_russian_plate_number.xml")
    # detect the number plates in the grayscale image
    detections = n_plate_detector.detectMultiScale(gray, scaleFactor=1.05, minNeighbors=7)

    # loop over the number plate bounding boxes
    for (x, y, w, h) in detections:
        # draw a rectangle around the number plate
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 255), 2)
        cv2.putText(image, "Number plate detected", (x - 20, y - 10),
                    cv2.FONT_HERSHEY_COMPLEX, 0.5, (0, 255, 255), 2)

        # extract the number plate from the grayscale image
        number_plate = gray[y:y + h, x:x + w]
        number_plate = cv2.resize(number_plate,(170,100)) #170,100

        text = tess.image_to_string(number_plate, lang='eng')
        text = text.strip()
        text = text.replace(" ", "")
        if(text.isalnum()==True):
            print("Number Plate: ",wd,text)
            for car in carpool:
                #print(car, text)
```

Fig. 6: Python code's snippet